

1.

$$f_7(n) = n^{1/\log n}$$

$$f_7(n) = (b^{\log_b n})^{1/\log_b n} = b^{\frac{\log_b n}{\log_b n}} = b^1 = b. \text{ Constant. Growth rate is } O(1)$$

$$f_2(n) = \sqrt{2n} \text{ simplifies to } \sqrt{2} \cdot \sqrt{n} = \sqrt{2} \cdot n^{0.5}. \text{ Growth rate is } O(n^{0.5})$$

$$f_3(n) = n + 10. \text{ Dominant term is } n. \text{ Growth rate } O(n)$$

$$f_6(n) = n^2 \log n. \text{ Polylogarithmic function. Growth rate is } O(n^2 \log n)$$

$$f_1(n) = n^{2.5}. \text{ Polynomial. Growth rate is } O(n^{2.5})$$

$$f_4(n) = 10^n. \text{ Exponential function with base 10. Growth rate is } O(10^n)$$

$$f_5(n) = 10^{2n} = (10^2)^n = 100^n. \text{ Growth rate is } O(100^n)$$

constant < polynomial < exponential

$$f_7(n), f_2(n), f_3(n), f_6(n), f_1(n), f_4(n), f_5(n)$$

2.

Algorithm

We can use DFS with node coloring. Each node $u \in V$ will be one of three states:

- a. White: Unvisited
- b. Gray: Currently visiting
- c. Black: Finished visiting

Initialize all nodes to white, then iterate through each vertex $u \in V$. If u is white, call DFS-Visit(u). If DFS-Visit(u) ever returns *true*, stop and return *true*. If the loop finishes, return *false*

DFS-Visit(u)

Mark u as gray.

For each neighbor v of u :

- If v is gray. A cycle exists. Return *true*

- If v is white. Recursively call DFS-Visit(v). If this call returns *true*, propagate it up by returning *true*.

Mark u as black

Return false

Running Time Analysis

- The main loop ensures that DFS-Visit is called exactly once for every vertex
- Inside DFS-Visit, the loop explores each edge outgoing from the current vertex
- Over the entire execution, each vertex is visited once, and each edge is explored exactly once
- Total running time is $O(|V| + |E|)$

Proof

- If the algorithm returns *true*, cycle exists. The algorithm only returns *true* when it explores an edge (u, v) and finds that v is gray only if it is an ancestor of u in the current DFS recursion stack. The path in the stack from v down to u , combined with the back edge (u, v) , forms a directed cycle.
- If a cycle exists, the algorithm returns *true*. Assume G has a cycle C . Let v be the first vertex in C that is discovered by DFS. When DFS-Visit(v) is called, it will eventually explore all nodes reachable from v . Since all nodes in C are reachable from v , the DFS will visit every other node in C before DFS-Visit(v) completes. Let (u, v) be the edge in the cycle C that points back to v . When DFS reaches u , u will be marked gray. The exploration u will eventually check the edge (u, v) .

v must still be gray. It cannot be black because DFS-Visit(v) is still running

It cannot be white because it was the first node in the cycle to be discovered. When the algorithm checks the edge (u, v) , it will see that v is gray, and it will correctly return *true*

3.

Analysis

A graph with no cycles is a forest. Remaining edges $T = E \setminus F$ form a forest. We want to build a spanning forest. We want to minimize the weight of removed edges F .

$$W(F) = W(E) - W(T)$$

Since $W(E)$ is constant, minimizing $W(F)$ is equivalent to maximizing $W(T)$. The set T we want to keep is a Maximum Spanning Forest.

Algorithm

Find a Maximum Spanning Forest T of the graph $G = (V, E)$. Altering Kruskal's algorithm

- Sort all edges E in descending order of weight
- Initialize $T = \emptyset$
- Initialize a Union-Find data structure for all $v \in V$
- Iterate through the sorted edges (u, v) from heaviest to lightest
- If $\text{Find}(u) \neq \text{Find}(v)$, add (u, v) to T and perform $\text{Union}(u, v)$

Return the set of edges $F = E \setminus T$

Running Time Analysis

- Sorting all $|E|$ edges takes $O(|E| \log |E|)$ time
- Initializing the Union find structure takes $O(|V|)$ time
- The loop iterates $|E|$ times. Each iteration performs two Find operations and one Union operation. Using a path compression and union by rank optimization, this takes $O(|E| \alpha(|V|))$ time, where α is a very slow growing inverse Ackermann function
- Dominant step is sorting. The total running time is $O(|E| \log |E|)$ or $O(|E| \log |V|)$, which is polynomial

Proof of Correctness

- No cycles if the algorithm returns $F = E \setminus T$, where T is the set of edges of a maximum spanning forest. By definition, a forest contains no cycles. Removing F from E results in the graph (V, T) , which has no cycles

- For minimum weight, we need to prove F has min weight. Let T_{max} be the max spanning forest found by our algorithm, and $F_{max} = E \setminus T_{max}$. Let T' be any spanning forest, and $F' = E \setminus T'$ be the corresponding set of removed edges
 - Our algo maxs the weight of the kept edges, $W(T_{max})$
 - Maximum spanning tree algo, $W(T_{max}) \geq W(T')$ for any other spanning forest T'
 - We want to minimize $W(F) = W(E) - W(T)$
 - Since $W(T_{max}) \geq W(T')$,

$$\begin{aligned}
 -W(T_{max}) &\leq -W(T') \\
 W(E) - W(T_{max}) &\leq W(E) - W(T') \\
 W(F_{max}) &\leq W(F')
 \end{aligned}$$
 - The weight of our removed set F_{max} is less than or equal to the weight of any other valid set of removed F' , meaning it is the minimum weight

4.

Analysis

Any path from s to t that uses the edge (u, v) must consist of three parts

- A path from s to u
- The edge (u, v) itself
- A path from v to t

$$L = \text{length}(s \rightarrow u) + w(u, v) + \text{length}(v \rightarrow t)$$

To minimize L , find the shortest path from s to u and the shortest path from v to t . Since all edge weights are non-negative, we can use Dijkstra's Algorithm

Algorithm

- Run Dijkstra's algorithm starting from source s on the graph G . This computes the shortest path distance $dist_s[x]$ from s to all other nodes $x \in V$
- Run Dijkstra's algorithm starting from source v on the graph G . This computes the shortest path distance $dist_v[x]$ from v to all other nodes $x \in V$
- Min path length is the sum: $L = dist_s[u] + w(u, v) + dist_v[t]$
- Return L

Running Time Analysis

- Dijkstra's algorithm, when implemented with a Fibonacci heap, runs in $O((|E| + |V|)\log|V|)$ or $O(|E|\log|V|)$
- We run Dijkstra's algorithm twice
- Total running time is dominated by these two runs, resulting in $O(|E| + |V|\log|V|)$, which is a polynomial

Proof of Correctness

Let P be any path from s to t passing through $e = (u, v)$. P can be decomposed into a subpath P_{su} from s to u , the edge e , and a

subpath P_{vt} from v to t . $length(P) = length(P_{su}) + w(e) + length(P_{vt})$

Let P^* be the shortest path from s to t through e . P^* must also be composed of a subpath P_{su}^* , the edge e , and a subpath P_{vt}^* .

$$length(P^*) = length(P_{su}^*) + w(e) + length(P_{vt}^*)$$

Due to optimal substructure property of shortest paths, the subpath P_{su}^* must be a shortest path from s to u , and P_{vt}^* must be a shortest path from v to t

- If P_{su}^* were not the shortest path from s to u , we could replace it with the true shortest path, P'_{su} , creating a new path

$P' = P'_{su} \rightarrow e \rightarrow P^*_{vt}$ with $length(P') < length(P^*)$,
contradiction

- Same logic applies to P^*_{vt}

Dijkstra's algorithm correctly finds the shortest path length from a source to all other nodes

- Step 1 computes $dist_s[u] = length(P^*_{su})$
- Step 2 computes $dist_v[t] = length(P^*_{vt})$. Therefore, the algorithm computes

$L = dist_s[u] + w(e) + dist_v[t] = length(P^*)$, which is the min length

5.

Connected $\rightarrow$ Cut Property

- Assume G is connected
- Assume for contradiction that there exists some nonempty proper subset $S \subset V$ such that no edge (u, v) exists with $u \in S$ and $v \in V \setminus S$
- $S' = V \setminus S$ since S is a nonempty proper subset, S' is also a nonempty proper subset
- Take any vertex $s \in S$ and any vertex $t \in S'$
- Because G is connected, there must exist a path P from s to t
- Path must start at $s \in S$ and end at $t \in S'$. Therefore, the path must "cross" the partition between S and S'
- Let the path be $P = \langle v_0, v_1, \dots, v_k \rangle$ where $v_0 = s$ and $v_k = t$
- Must be some index $j (0 \leq j < k)$ such that $v_j \in S$ and $v_{j+1} \in S'$
- The edge $e = (v_j, v_{j+1})$ is in E. The edge has one endpoint in S and one endpoint in $V \setminus S$
- Contradicts our assumption that no such edge exists
- Therefore, if G is connected, the cut property must hold

Cut Property $\rightarrow$ Connected

- Assume the cut property holds, for every nonempty proper subset $S \subset V$, there is an edge crossing $(S, V \setminus S)$
- Assume for contradiction that G is not connected
- If G is not connected, it must have at least two connected components.
- Let S be the set of all vertices in *one* of these connected components.
- Since G is not connected, S must be a proper subset of V
- Since components are nonempty, S is nonempty.
- By the definition of a connected component, there are no edges between any vertex in S and any vertex not in S
- This violates our assumption that the cut property holds for every nonempty proper subset S
- This is a contradiction.
- Therefore, if the cut property holds, G must be connected

6.

Proof of Existence

Consider a maximal independent set I . An independent set is a set of vertices where no two are adjacent. A maximal independent set is one where no more vertices can be added

Claim 1: I is a dominating set

- Let v be any vertex in V
- If $v \in I$, then by definition of a maximal independent set, v must be adjacent to some vertex $u \in I$. v has a neighbor in I
- Therefore, I is a dominating set

Claim 2: $S' = V \setminus I$ is also a dominating set

- Let v be any vertex in V
- If $v \in S'$, it is dominated
- If $v \in I$, then v cannot have any neighbors in I . Since the graph is connected, v must have at least one neighbor. That neighbor must be in $V \setminus I = S'$

- Therefore, S' is also a dominating set

Now have two dominating sets, I and $S' = V \setminus I$. We know that

$$|I| + |S'| = |V|$$

Following pigeonhole principle, at least one of these two sets must have size less than or equal to $|V|/2$. Let S be the smaller of the two sets I and S' . Then S is a dominating set and $|S| \leq |V|/2$

Algorithm

The proof gives us the algorithm, find a maximal independent set I , and then choose the smaller of I and $V \setminus I$

- Initialize $I = \emptyset$
- Create a boolean array `is_dominated` of size $|V|$, initialized to *false*
- Iterate through each vertex $v \in V$
- If `is_dominated[v]` is *false*
 - Add v to I
 - Mark `is_dominated[v] = true`
 - For each neighbor u of v
 - Mark `is_dominated[u] = true`
- At this point, I is a maximal independent set, and `is_dominated` marks all vertices in I or adjacent to I
- If $|I| \leq |V|/2$, return I
- Else, compute $S' = V \setminus I$ and return S'

Running Time Analysis

- Steps 1 and 2 take $O(|V|)$ time
- The loop in Step 3 runs $|V|$ times
- The inner loop (Step 4) processes each vertex and its neighbors. A vertex v is added to I only once. Its adjacency list is explored once. Every edge (u, v) is examined at most twice
- The total time for steps 3-5 is $O(|V| + |E|)$
- Step 6 takes $O(1)$
- Step 7 takes $O(|V|)$ to build the complement set S'

- Overall running time is dominated by the graph traversal, making $O(|V| + |E|)$